

Встановлення докера та запуск контейнера

Основні компоненти Docker

Docker Engine — це ядро Docker, яке керує контейнерами та образами.

Docker Images (образи Docker) — це шаблони для створення контейнерів, що містять додаток і всі його залежності.

Docker Containers (контейнери Docker) — це ізольовані середовища, створені на основі образів, які виконуються як екземпляри додатків Docker.

Docker Compose — спрощує налаштування додатків, що потребують кількох контейнерів, дозволяючи керувати ними як єдиною системою.

Встановіть Docker Engine на Ubuntu

Щоб розпочати роботу з Docker Engine в Ubuntu, переконайтеся, що ви відповідаєте попереднім вимогам, а потім інсталюйте Docker.

Необхідні умови: вимоги до ОС

Щоб встановити **Docker Engine**, вам потрібна 64-бітна версія однієї з таких операційних систем:

- **Ubuntu LTS**
- **Debian LTS**
- **Red Hat Enterprise Linux (RHEL) LTS**
- **Fedora LTS**

Docker Engine підтримує архітектури **x86_64 (або amd64)**, **armhf** та **arm64**.

Видаліть старі версії

Старіші версії Docker називалися **docker**, **docker.io** або **docker-engine**. Якщо вони встановлені, видаліть їх:

```
$ sudo apt-get remove docker docker-engine docker.io containerd runc
```

Це нормально, якщо **apt-get** повідомляє, що жоден із цих пакетів не встановлено.

Вміст **/var/lib/docker/**, включаючи зображення, контейнери, томи та мережі, зберігається. Якщо вам не потрібно зберігати наявні дані та ви хочете почати з чистої інсталяції, зверніться до розділу видалення Docker Engine внизу цієї сторінки.

Підтримувані драйвери накопичувачів

Docker Engine на Ubuntu підтримує драйвери зберігання **overlay2**, **aufs** і **btrfs**.

Docker Engine за замовчуванням використовує драйвер зберігання **overlay2**. Якщо замість цього вам потрібно використовувати **aufs**, вам потрібно налаштувати його вручну.

Способи встановлення

Ви можете встановити Docker Engine різними способами, залежно від ваших потреб:

- Більшість користувачів налаштовують репозиторії Docker і встановлюють із них, щоб спростити встановлення й оновлення. Це рекомендований підхід.
- Деякі користувачі завантажують пакет DEB і встановлюють його вручну, а також повністю керують оновленнями вручну. Це корисно в таких ситуаціях, як встановлення Docker на системах із зазором без доступу до Інтернету.
- У середовищах тестування та розробки деякі користувачі вирішують використовувати автоматизовані зручні сценарії для встановлення Docker.

Встановити за допомогою репозиторію

Перш ніж вперше встановлювати Docker Engine на нову хост-машину, вам потрібно налаштувати репозиторій Docker. Після цього ви можете встановити та оновити Docker зі сховища.

НАЛАШТУВАТИ РЕПОЗИТОРІЙ

1. Оновіть індекс пакетів apt і встановіть пакунки, щоб дозволити apt використовувати репозиторій через HTTPS:

```
$ sudo apt-get update

$ sudo apt-get install \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg-agent \
    software-properties-common
```

Встановити за допомогою репозиторію

2. Додайте офіційний ключ GPG Docker:

```
$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Перевірте, чи зараз у вас є ключ із відбитком пальця **9DC8 5822 9FC7 DD38 854A E2D8 8D81 803C 0EBF CD88**, виконавши пошук за останніми 8 символами відбитка пальця.

```
$ sudo apt-key fingerprint 0EBFCD88

pub   rsa4096 2017-02-22 [SCEA]
      9DC8 5822 9FC7 DD38 854A  E2D8 8D81 803C 0EBF CD88
uid           [ unknown] Docker Release (CE deb) <docker@docker.com>
sub   rsa4096 2017-02-22 [S]
```

Встановити за допомогою репозиторію

3. Використовуйте наступну команду, щоб налаштувати стабільний репозиторій. Щоб додати нічний або тестовий репозиторій, додайте слово `nightly` або `test` (або обидва) після слова `stable` у наведених нижче командах.

```
$ sudo add-apt-repository \  
    "deb [arch=amd64] https://download.docker.com/linux/ubuntu \  
    $(lsb_release -cs) \  
    stable"
```

ВСТАНОВИТИ DOCKER ENGINE

1. Оновіть індекс пакетів apt і встановіть останню версію Docker Engine і контейнера або перейдіть до наступного кроку, щоб установити певну версію:

```
$ sudo apt-get update  
$ sudo apt-get install docker-ce docker-ce-cli containerd.io
```

2. Щоб установити певну версію Docker Engine, перелічіть доступні версії в сховищі, а потім виберіть і встановіть:
 - а. Перелічіть версії, доступні у вашому репо:

```
$ apt-cache madison docker-ce  
  
docker-ce | 5:18.09.1~3-0~ubuntu-xenial | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages  
docker-ce | 5:18.09.0~3-0~ubuntu-xenial | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages  
docker-ce | 18.06.1~ce~3-0~ubuntu | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages  
docker-ce | 18.06.0~ce~3-0~ubuntu | https://download.docker.com/linux/ubuntu | xenial/stable amd64 Packages  
...
```

ВСТАНОВИТИ DOCKER ENGINE

б. Встановіть певну версію, використовуючи рядок версії з другого стовпця, наприклад, 5:18.09.1~3-0~ubuntu-xenial.

```
$ sudo apt-get install docker-ce=<VERSION_STRING> docker-ce-cli=<VERSION_STRING> containerd.io
```

3. Переконайтеся, що Docker Engine встановлено правильно, запустивши образ hello-world.

```
$ sudo docker run hello-world
```

Ця команда завантажує тестове зображення та запускає його в контейнері. Коли контейнер працює, він друкує інформаційне повідомлення та виходить.

ОНОВЛЕННЯ DOCKER ENGINE

Docker Engine встановлено та працює. Групу докерів створено, але до неї не додано жодного користувача. Для запуску команд Docker потрібно використовувати `sudo`. Перейдіть до Linux після інсталяції, щоб дозволити непривілейованим користувачам запускати команди Docker та виконувати інші додаткові кроки налаштування.

Щоб оновити Docker Engine, спочатку запустіть `sudo apt-get update`, а потім дотримуйтесь інструкцій із встановлення, вибравши нову версію, яку хочете встановити.

Постінсталяційні дії для Linux

Цей розділ містить додаткові процедури для налаштування хостів Linux для кращої роботи з Docker

Керуйте Docker як не головний користувач

Демон Docker прив'язується до сокету Unix замість порту TCP. За замовчуванням цей сокет Unix належить користувачеві root, і інші користувачі можуть отримати до нього доступ лише за допомогою sudo. Демон Docker завжди працює як користувач root.

Якщо ви не хочете перед командою docker використовувати sudo, створіть групу Unix під назвою docker і додайте до неї користувачів. Коли демон Docker запускається, він створює сокет Unix, доступний членам групи докерів.

Керуйте Docker як не головний користувач

Щоб створити групу докерів і додати свого користувача:

1. Створіть групу докерів. `$ sudo groupadd docker`
2. Додайте свого користувача до групи докерів. `$ sudo usermod -aG docker $USER`
3. Вийдіть і увійдіть знову, щоб повторно оцінити ваше членство в групі. У разі тестування на віртуальній машині може знадобитися перезапустити віртуальну машину, щоб зміни набули чинності. У настільному середовищі Linux, наприклад X Windows, повністю вийдіть із сеансу, а потім увійдіть знову. У Linux ви також можете виконати таку команду, щоб активувати зміни в групах:

```
$ newgrp docker
```

Керуйте Docker як не головний користувач

4. Переконайтеся, що ви можете запускати команди докерів без sudo.

```
$ docker run hello-world
```

Ця команда завантажує тестове зображення та запускає його в контейнері. Коли контейнер працює, він друкує інформаційне повідомлення та виходить.

Якщо ви спочатку запускали команди Docker CLI за допомогою sudo перед додаванням свого користувача до групи docker, ви можете побачити таку помилку, яка вказує на те, що ваш каталог ~/.docker/ було створено з неправильними дозволами через команди sudo.

```
WARNING: Error loading config file: /home/user/.docker/config.json -  
stat /home/user/.docker/config.json: permission denied
```

Керуйте Docker як не головний користувач

Щоб вирішити цю проблему, або видаліть каталог ~/.docker/ (він буде створений автоматично, але будь-які користувацькі налаштування втрачено), або змініть його власника та дозволи за допомогою таких команд:

```
$ sudo chown "$USER":"$USER" /home/"$USER"/.docker -R
$ sudo chmod g+rx "$HOME/.docker" -R
```

Налаштуйте Docker для запуску під час завантаження

Більшість сучасних дистрибутивів Linux (RHEL, CentOS, Fedora, Ubuntu 16.04 і вище) використовують systemd для керування тим, які служби запускаються під час завантаження системи. Ubuntu 14.10 і нижче використовують upstart.

systemd

```
$ sudo systemctl enable docker
```

Щоб вимкнути таку поведінку, використовуйте disable.

```
$ sudo systemctl disable docker
```

Налаштуйте Docker для запуску під час завантаження

upstart

Docker автоматично налаштовується на запуск під час завантаження за допомогою upstart. Щоб вимкнути цю поведінку, скористайтеся такою командою:

```
$ echo manual | sudo tee /etc/init/docker.override
```

chkconfig

```
$ sudo chkconfig docker on
```

Використовуйте інший механізм зберігання

Інформацію про різні механізми зберігання див. у розділі Драйвери зберігання. Механізм зберігання за замовчуванням і список підтримуваних механізмів зберігання залежать від дистрибутива Linux на вашому хості та доступних драйверів ядра.

Налаштувати драйвер журналювання за замовчуванням

Docker надає можливість збирати та переглядати дані журналу з усіх контейнерів, що працюють на хості, за допомогою серії драйверів журналювання. Драйвер журналювання за замовчуванням, **json-file**, записує дані журналу у файли у форматі JSON у файловій системі хоста. З часом ці файли журналу збільшуються в розмірі, що призводить до потенційного виснаження дискових ресурсів. Щоб усунути такі проблеми, або налаштуйте альтернативний драйвер журналювання, наприклад Splunk або Syslog, або налаштуйте ротацію журналу для драйвера за замовчуванням.

Налаштуйте, де Docker daemon прослуховує підключення

За замовчуванням Docker daemon прослуховує з'єднання в сокеті UNIX, щоб приймати запити від локальних клієнтів. Можна дозволити Docker приймати запити від віддалених хостів, налаштувавши його на прослуховування IP-адреси та порту, а також сокета UNIX.

Налаштування Docker для прийняття віддалених з'єднань можна виконати за допомогою файлу модуля `docker.service` `systemd` для дистрибутивів Linux із використанням `systemd`, таких як останні версії RedHat, CentOS, Ubuntu і SLES, або за допомогою файлу `daemon.json`, який рекомендується для дистрибутивів Linux, які не використовують `systemd`.

Налаштування віддаленого доступу за допомогою файлу модуля systemd

1. Використовуйте команду `sudo systemctl edit docker.service`, щоб відкрити файл заміни для `docker.service` у текстовому редакторі.
2. Додайте або змініть наступні рядки, замінивши власні значення.

```
[Service]
ExecStart=
ExecStart=/usr/bin/dockerd -H fd:// -H tcp://127.0.0.1:2375
```

Налаштування віддаленого доступу за допомогою файлу модуля systemd

3. Збережіть файл.

4. Перезавантажте конфігурацію systemctl.

```
$ sudo systemctl daemon-reload
```

5. Перезапустіть Docker.

```
$ sudo systemctl restart docker.service
```

6. Перевірте, чи було враховано зміну, переглянувши вихідні дані netstat, щоб підтвердити, що dockerd прослуховує налаштований порт.

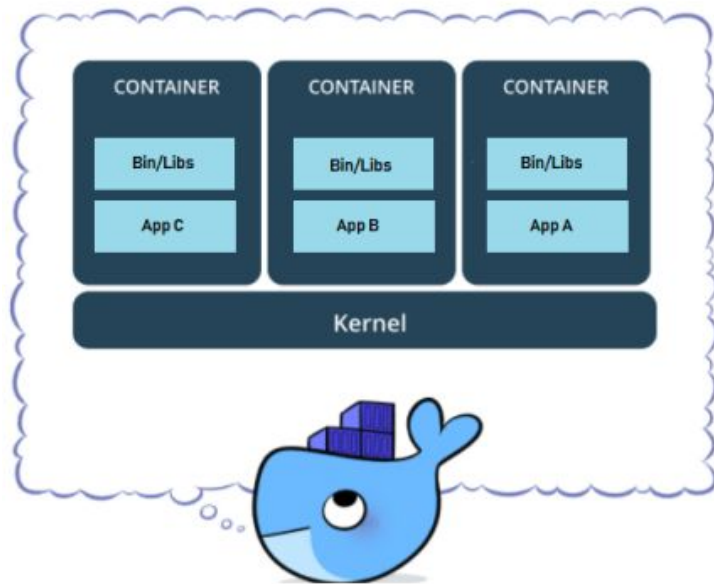
```
$ sudo netstat -lntp | grep dockerd
```

tcp	0	0	127.0.0.1:2375	0.0.0.0:*	LISTEN	3758/dockerd
-----	---	---	----------------	-----------	--------	--------------

Вступ до мереж Docker

Мережі Docker дозволяють контейнерам спілкуватися між собою та із зовнішнім світом.

Вони забезпечують **ізоляцію** та **керування мережею** за допомогою спеціальних **мережесих драйверів** (network drivers).



Типи мереж Docker

Bridge (Міст): стандартний тип мережі для контейнерів, що дозволяє контейнерам спілкуватися між собою на одному хості.

Host (Хост): видаляє мережеву ізоляцію між контейнером і хостом Docker, дозволяючи контейнерам використовувати мережу хоста напряму.

Overlay (Накладна мережа): з'єднує кілька демонів Docker і дозволяє службам Swarm взаємодіяти між собою.

Macvlan: дозволяє присвоїти контейнеру MAC-адресу, завдяки чому він виглядає як фізичний пристрій у вашій мережі.

Мережа Docker Bridge

Мережа **Bridge** — це типовий (за замовчуванням) тип мережі при запуску контейнера. Вона дозволяє контейнерам, підключеним до однієї мережі мосту, спілкуватися між собою за IP-адресами. Docker автоматично присвоює кожному контейнеру приватну IP-адресу, щоб уникнути конфліктів із мережею хоста.

Основні характеристики:

- **Ізоляція:** забезпечує високий рівень ізоляції — контейнери за замовчуванням не можуть взаємодіяти з хостом, якщо не проброшено конкретні порти.
- **Простота налаштування:** Docker налаштовує мережу автоматично, додаткових дій зазвичай не потрібно, якщо не потрібні спеціальні конфігурації.
- **Використання:** ідеально підходить для запуску кількох контейнерів на одному хості, які потребують взаємодії між собою, при цьому залишаючись ізольованими від мережі хоста.

Команди:

Створити власну bridge-мережу:

```
docker network create --driver bridge [network_name]
```

Запустити контейнер у певній мережі:

```
docker run --network=[network_name] [other_options] [image]
```

Мережа Docker Host

Мережа **Host** обминає керовану Docker мережеву інфраструктуру і дозволяє контейнерам використовувати мережу хоста напряду. Це означає, що кожен порт, відкритий у контейнері, доступний на IP-адресі хоста.

Основні характеристики:

- **Продуктивність:** забезпечує найвищу мережеву продуктивність, оскільки немає ізоляції між хостом і контейнером, що знижує накладні витрати.
- **Прямий доступ:** контейнери можуть напряду використовувати IP хоста, що спрощує обробку вхідного трафіку без пробросу портів.
- **Безпека:** менш безпечно порівняно з іншими типами мереж, оскільки контейнери ділять мережевий простір хоста, потенційно підвищуючи ризик для системи хоста.
- **Використання:** підходить для додатків, що потребують реального часу або високопродуктивних мережевих з'єднань, де критично важлива мінімальна затримка та максимальна швидкість.

Команда запуску контейнера з мережею хоста:

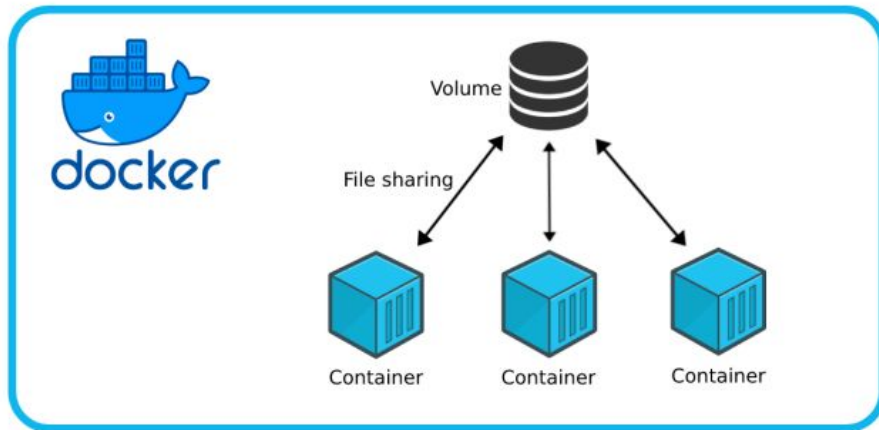
```
docker run --network host [other_options] [image]
```

Вступ до Docker Volumes

Docker Volumes — це механізм для зберігання **постійних даних** у Docker.

Вони **зберігають дані навіть після перезапуску контейнера**, відокремлюючи життєвий цикл даних від життєвого циклу контейнера.

Volumes забезпечують **ефективний і безпечний спосіб управління даними**, що дозволяє зручно працювати з базами даних, файлами конфігурацій та іншими важливими даними всередині контейнерів.



Типи зберігання даних у Docker

Volumes (Томи): зберігаються у частині файлової системи хоста, якою керує Docker (наприклад, `/var/lib/docker/volumes/` на Linux).

Bind mounts (Прив'язані монтування): можуть зберігатися у будь-якому місці на хості і часто використовуються для специфічних потреб або старих додатків.

Tmpfs mounts (Тимчасові монтування): зберігаються лише в оперативній пам'яті хоста, ідеально підходять для чутливої інформації або тимчасових даних, що не потребують збереження після перезапуску контейнера.

Створення та керування Docker Volumes

- **Volumes** є важливою складовою контейнеризованого середовища. Вони надзвичайно потужні та гнучкі, забезпечують **постійне зберігання даних між контейнерами** і навіть після їх видалення.
- Вміння створювати та керувати томами є ключем до ефективного використання Docker. Існують різні стратегії для створення, налаштування та ефективного управління томами, що дає повний контроль над вашими даними.

Основні команди для роботи з томами:

- Створити том:

```
docker volume create [volume_name]
```

- Переглянути список томів:

```
docker volume ls
```

- Видалити том:

```
docker volume rm [volume_name]
```

Використання Volumes з Docker контейнерами

- **Volumes** забезпечують **безшовну інтеграцію** з контейнерами Docker, дозволяючи легко керувати **постійністю даних** і їхнім **обміном між контейнерами**. Завдяки томам додатки мають доступ до необхідних даних незалежно від життєвого циклу контейнера.
- **Приклад:** якщо у вас є Docker-контейнер із базою даних і ви хочете, щоб дані бази зберігалися навіть після зупинки або видалення контейнера, можна використати команду:

```
docker run -d --name my_database \  
  
-v /path/to/database/data:/var/lib/mysql \  
  
-e MYSQL_ROOT_PASSWORD=password \  
  
mysql:latest
```

♦ `-v /path/to/database/data:/var/lib/mysql` — монтує локальну директорію хоста в контейнер, забезпечуючи збереження даних бази поза контейнером.

Питання та відповіді